Bagging, Boosting and Bias–Variance Measurements for Face Identification with Decision Trees on Olivetti Faces

GEETIKA SINGH
Roll No.: M26CPS003
Repository: https://github.com/geetika-singh04/ML-assignment
Final commit: 8f99a2a5980d9ece15ad9477cd017be07a42fc6f

Abstract

We study decision trees, bagging and multiclass boosting (SAMME) on the Olivetti Faces data (40 identities, raw 64×64 pixels, no dimensionality reduction). Bagging and boosting are implemented from scratch. A single unlimited-depth tree reaches 0.562 test accuracy; bagging with 100 trees reaches 0.863, and SAMME with depth-4 trees and 100 rounds reaches 0.575. Occluding the central 16×16 block cuts single-tree accuracy to 0.075 and bagging to 0.588, while boosting stays at 0.575 only because 5 correct and 5 incorrect predictions flip. Over 30 resampled training sets, bagging removes 96% of the single-tree variance at unlimited depth, but the largest accuracy gain occurs at depth 8, not at the deepest tree.

1. Setup

Data and split. We use `fetch_olivetti_faces`: 400 images, 40 identities, 10 images each, flattened to 4096 raw pixel values. The seed for `numpy.random.default_rng` is formed from all digits of the roll number, in order. Within each identity the 10 images are randomly permuted and split 6/2/2 into train/validation/test (240/80/80 images). The split is saved in `split.npz` and is used by every experiment. All model selection uses validation accuracy only; the test sets are never used for choices. Since the validation and test sets have 80 images, one image equals 0.0125 accuracy, so differences of one or two images are within noise.

Masked test set. A second test set is created by setting rows and columns 24–39 (the centred 16×16 block) to zero. All models are trained on clean images only.

Models. Single trees use `DecisionTreeClassifier`. Bagging and SAMME are implemented by us on top of it. Ties in selection go to the first setting in the grid.

Depth	Train	Val	Leaves	Samples/leaf	1-ex. leaves
2	0.075	0.075	3	80.00	0.000
4	0.125	0.100	5	48.00	0.000
8	0.225	0.188	9	26.67	0.000
16	0.425	0.287	17	14.12	0.000
None	1.000	0.562	44	5.45	0.091

Table 1. Single-tree grid (leaf $= 1$). The frozen tree (depth None, leaf 1) has test accuracy 0.562.

2. Q1: Single tree and bagging

2.1. (a) Single-tree baseline

We trained trees for depth $\in \{2, 4, 8, 16, \text{None}\}$ and min_samples_leaf $\in \{1, 2, 4\}$. Table 1 gives results for leaf $= 1$. For depth ≤ 16 the three leaf values gave identical accuracies. For unlimited depth, validation accuracy was 0.562 (leaf 1), 0.562 (leaf 2) and 0.537 (leaf 4). The first setting in the grid among the best, **depth=None, leaf=1**, was frozen.

What changes with depth. The trees grow as a chain: for depths up to 16 the number of leaves is depth$+1$ (3, 5, 9, 17), because each split separates off one group of images and the tree continues on the other side. Each leaf can therefore fit about one identity, and training accuracy equals $0.025\times$ the number of leaves (each identity has $6/240 = 2.5\%$ of the training images): 0.075, 0.125, 0.225, 0.425. Mean samples per leaf falls from 80.0 to 14.1, and no leaf holds a single image until the depth limit is removed. Since 40 identities need at least 40 leaves, only the unlimited tree reaches 1.000 training accuracy. It has depth 37 and 44 leaves, only slightly more than the 40 identities, with 5.45 images per leaf on average, and 9.1% of its leaves (4 of 44) hold a single training image. Validation accuracy (0.562) stays far below training accuracy: the tree memorises the six images of each identity and generalises poorly to new images. This also explains why `min_samples_leaf` had no effect for depth ≤ 16: every leaf already had at least 14

B	Train	Val	Test	Distinct	OOB
1	0.750	0.287	0.325	152.0	0.318
5	0.967	0.487	0.487	151.0	0.317
10	1.000	0.650	0.750	152.7	0.451
25	1.000	0.750	0.800	152.0	0.658
50	1.000	0.825	0.812	151.9	0.758
100	1.000	0.838	0.863	152.4	0.808
200	1.000	0.825	0.863	152.3	0.829

Table 2. Bagging versus B. "Distinct" is the average number of distinct training images per bootstrap sample.

images.

2.2. (b) Bagging

We draw B bootstrap samples of size $n = 240$ with replacement and train one frozen tree on each. Models for different B are nested (the first B of 200 trees), and prediction is by plurality vote. Table 2 reports results.

Distinct examples. A given training image is missed by one bootstrap sample with probability $(1 - 1/n)^n$, so the expected number of distinct images is $n\,[1-(1-1/n)^n] = 151.89$ ($\approx 0.633\,n$, tending to $0.632\,n$). The measured averages (151.0–152.7) agree with this: the largest gap is 0.9 images, and the value for $B = 200$ is 152.3. Each bootstrap sample therefore leaves out about 36.8% of the training images, which is what OOB evaluation uses.

2.3. (c) OOB accuracy, disagreement and vote margin

OOB accuracy is computed for each training image using only trees whose bootstrap sample did not contain it (images with no OOB tree are skipped). Disagreement is the average pairwise disagreement between trees' predictions, and the vote margin is the winning vote fraction minus the runner-up fraction; both are computed on the validation set. Figure 1 shows all three against B.

Choice of B. Validation accuracy is highest at $B = 100$ (0.838) and 0.825 at $B = 200$, so $B = \mathbf{100}$ is selected. The difference is one validation image, so we regard 100 and 200 as equivalent.

Do the three measures agree? Not entirely. (i) OOB accuracy keeps rising up to $B = 200$ (0.829), while validation accuracy saturates near 0.83 after $B = 50$. OOB is low for small B because each image is out-of-bag for only $0.368B$ trees (about 3.7 trees at $B = 10$), so its vote is noisy; it measures the ensemble on a smaller effective size. (ii) The mean vote margin grows from 0.19 ($B = 5$) to 0.263 ($B = 25$) and plateaus at about 0.27 from $B = 50$. Margins are small in absolute terms because the 40-class votes are spread out. The margin at $B = 1$ equals 1 by construction. (iii) Disagreement stays at about 0.80 for every $B \geq 5$: it describes how different the trees are, which does not depend on how many we average. So OOB, validation accuracy and margin all say that more trees help up to about 50–100 and then stop mattering much, while disagreement says that the diversity of the trees is constant, and that this diversity is what makes averaging effective.

	Accuracy		Confidence	
Model	Clean	Masked	Clean	Masked
Single tree	0.562	0.075	1.000	1.000
Bagging (B=100)	0.863	0.588	0.415	0.276

Table 3. Test accuracy and mean prediction confidence (tree: predicted probability; bagging: winning vote fraction).

Case	Test idx	True	Tree	Bag	Votes (true / bag)
Bag corrects	2	1	37	1	33 / 33
Bag corrects	5	2	3	2	33 / 33
Both wrong	9	4	8	2	3 / 20
Both wrong	16	8	22	39	9 / 11

Table 4. Clean-test cases. Votes are out of 100 trees.

2.4. (d) Tree versus bagging, clean versus masked

The single tree is entirely confident (1.000) on both test sets, even though its accuracy collapses from 0.562 to 0.075 on masked faces: unlimited-depth leaves are pure, so the confidence carries no information. Bagging's confidence falls from 0.415 to 0.276 together with accuracy, so its votes are a more honest signal.

Interpretation. In the two corrected cases, the true identity receives a plurality of 33 of 100 votes, though not a majority. Individual trees are unstable: a different bootstrap sample leads to a different split, so the single frozen tree went to the wrong identity, while averaging cancels these idiosyncratic splits. This is a *variance* error, which bagging reduces. In the two failures, the true identity gets only 3 and 9 votes, and the winning identity is not strongly supported either (20 and 11 votes). Most trees make the same mistake or scatter their votes over wrong identities, so the face is hard to separate in raw pixels (for example because of expression or pose changes relative to the six training images). This is a *bias* or inherent-difficulty error, which averaging does not remove.

3. Q2: Multiclass boosting (SAMME)

3.1. (a) A complete round on a toy problem

We implemented SAMME with the weak-learner weight $\alpha_t = \log\frac{1-\epsilon_t}{\epsilon_t} + \log(K-1)$, the update $w_i \leftarrow w_i \exp(\alpha_t \mathbb{1}[h_t(x_i) \neq y_i])$ followed by normalisation, and the normalised vote $q_k(x) = \sum_t \alpha_t \mathbb{1}[h_t(x) = k] / \sum_t \alpha_t$. A weak learner whose weighted error is at least $1-1/K$ (no

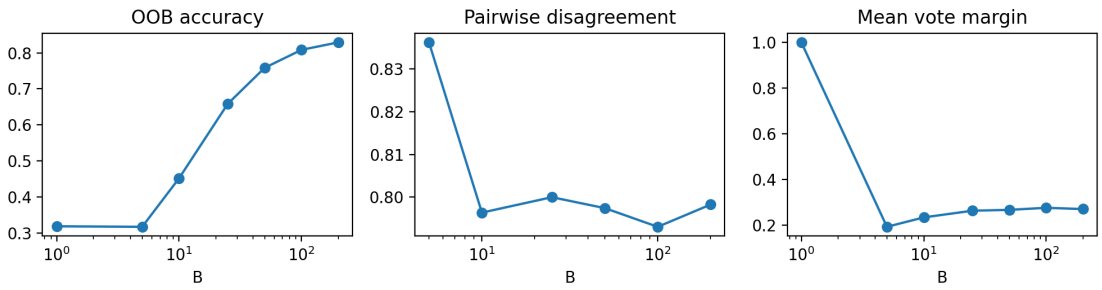


Figure 1. OOB accuracy, pairwise disagreement and mean vote margin versus B (log scale).



Figure 2. The four faces of Table 4.

better than random guessing) would have $\alpha_t \leq 0$; the implementation rejects it and stops boosting. A perfect learner ($\epsilon_t = 0$) is clipped to $\epsilon = 10^{-10}$ and also stops the loop. Neither stopping rule was triggered on the face data (all selected models ran the full number of rounds). The same function is used for the toy problem and the face data.

Toy problem: $K = 3$, 9 one-dimensional samples with labels $[0, 0, 0, 1, 1, 0, 2, 2, 2]$ and a depth-1 tree. Initial weights are $1/9$ each. The stump predicts $[0, 0, 0, 0, 0, 0, 2, 2, 2]$, misclassifying the two samples with label 1, so $\epsilon = 2/9 = 0.2222$ and $\alpha = \ln 3.5 + \ln 2 = 1.9459$. The updated weights are 0.0476 for each of the 7 correct samples and 0.3333 for each of the 2 misclassified samples (sum $= 1$). Note that after the update the misclassified samples carry total weight exactly $(K-1)/K = 2/3$, the weight of random guessing, so the next learner must focus on them.

3.2. (b) Depth and number of rounds

Table 5 shows validation accuracy for weak-learner depths 1–4 and 10, 25, 50, 100 rounds (nested prefixes of one 100-round run per depth). The best validation accuracy, 0.463, is for **depth 4, 100 rounds**, which has training accuracy 1.000. This is the edge of the grid and validation accuracy is still rising, so a larger grid might choose differently.

Observations. The weighted error of the first learner is high (about 0.85) and drops sharply to about 0.4 at the second to third round, which is also where α_t peaks (about

Depth	10	25	50	100
1	0.037	0.050	0.037	0.125
2	0.100	0.188	0.175	0.225
3	0.175	0.200	0.225	0.312
4	0.138	0.350	0.438	**0.463**

Table 5. Validation accuracy of SAMME by weak-learner depth (rows) and rounds (columns).

4). After that, the weighted error fluctuates between about 0.55 and 0.7 for the remaining rounds, always well below the random-guessing limit $1 - 1/40 = 0.975$, and α_t stays around 3, rising and falling inversely to the error. The maximum sample weight and the total weight of the 10 heaviest images both rise over the first $\sim$30 rounds (maximum weight from $\approx$0.004 to 0.04–0.08, top-10 weight to 0.2–0.4) and then fluctuate: a small group of hard faces repeatedly takes a large share of the total weight.

3.3. (c) Boosting versus bagging

Bagging is clearly better here: at 100 learners it reaches 0.887 test accuracy against 0.575. Boosting is cheaper per learner, because its weak trees have depth 4, while bagging's trees are fully grown (100.5 s vs. 338.7 s at 100 learners, so about 3.4 times slower). With 40 classes and six training images per class, each depth-4 tree can only separate a few identities, and boosting has to chain many such

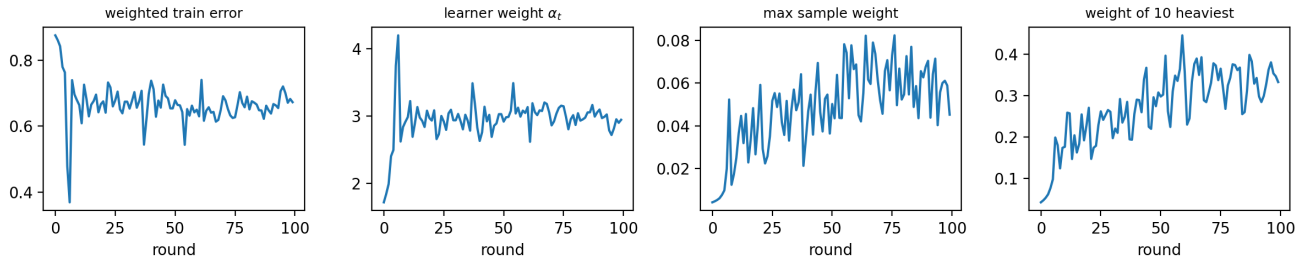


Figure 3. Selected boosting model (depth 4, 100 rounds): weighted training error, learner weight α_t, maximum sample weight and total weight of the 10 heaviest training images over rounds.

Method	n	Train	Val	Test	Time (s)
Boosting	25	0.854	0.350	0.362	26.5
Bagging	25	1.000	0.775	0.800	84.8
Boosting	50	0.942	0.438	0.412	50.9
Bagging	50	1.000	0.800	0.875	171.6
Boosting	100	1.000	0.463	0.575	100.5
Bagging	100	1.000	0.812	0.887	338.7

Table 6. Boosting (depth 4) versus bagging (unlimited-depth trees) with n learners. The bagging rows were trained here with their own bootstrap seed, so the 100-tree row (0.887) differs slightly from the Q1 model (0.863).

trees; its accuracy was still increasing at 100 rounds.

A heavily weighted face. Training face 158 (identity 26) reaches a weight of 0.0823, about $19.6\times$ the uniform weight $1/240 = 0.0042$, and is misclassified by 87 of the 100 learners. The weight trajectory (Fig. 4) shows the mechanism. Before round 3 its weight grows from 0.0042 to 0.0061. At round 3 the learner classifies it correctly, and by the next round its weight drops about nine-fold (0.0007). It is then missed repeatedly, its weight climbs back (0.0009, 0.0018, 0.0047, 0.0087, 0.0126), and the learner at round 10 classifies it correctly again before it is missed again. SAMME multiplies the weight of every misclassified example by e^{α_t} and renormalises, so an example that is repeatedly misclassified is multiplied again and again, which forces the next tree to pay attention to it. A learner that fixes it must take capacity from other examples. Since depth-4 trees cannot separate 40 identities, fixing one face breaks others, and the face is missed again in the next rounds: attention returns to it cyclically.

3.4. (d) Clean versus masked test sets

Boosting has the same accuracy (0.575) on the clean and masked sets, and its mean winning weight only drops from 0.112 to 0.107. This is not evidence of robustness: masking changes 18 of its 80 test predictions, 5 from correct to wrong and 5 from wrong to correct, and these cancel

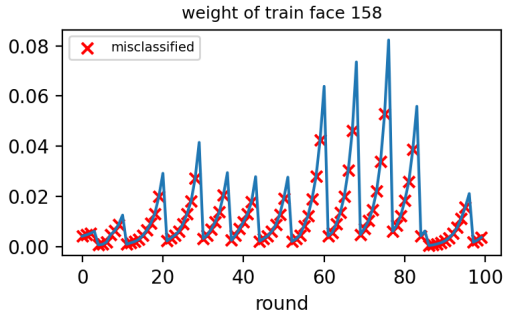


Figure 4. Weight of training face 158 over rounds. Red crosses mark rounds where it is misclassified.

	Accuracy		Mean winning prob./vote	
Model	Clean	Masked	Clean	Masked
Single tree	0.562	0.075	1.000	1.000
Bagging (B=100)	0.863	0.588	0.415	0.276
Boosting (d4, 100)	0.575	0.575	0.112	0.107

Table 7. Frozen models on the clean and masked test sets.

exactly. For comparison, masking changes 31 of 80 bagging predictions (22 correct→wrong, none wrong→correct) and 75 of 80 single-tree predictions (40 correct→wrong, 1 wrong→correct). Boosting's predictions are the most stable under the mask, but not because it avoids the masked pixels: 6.0% of its splits use pixels in the block, against 7.0% for the single tree and 7.1% for bagging, close to the block's 6.25% share of the image. A plausible explanation, which we did not test, is depth: a boosted tree has about 7.5 splits (748 splits over 100 trees), while a bagged tree has about 50, and a wrong turn early in a deep tree sends the face to a wrong identity for good, whereas one flipped shallow learner changes only a small part of the weighted vote.

A face that reacts differently. Test face 2 (identity 1) is classified correctly by bagging on the clean image (33 votes for the true identity) but incorrectly after mask-

Model	Test acc. (mean ± std)	Bias^2	Var
Single tree	0.488 ± 0.042	0.394	0.630
Bagging	0.775 ± 0.034	0.493	0.025
Boosting	0.350 ± 0.082	0.883	0.017

Table 8. Clean test accuracy over 30 refits, with measured bias^2 and variance.

ing (predicted identity 13, winning vote only 0.170). The pairwise disagreement of the 100 trees on the masked face is 0.920, so the trees disagree almost completely:a zeroed pixel changes the branch taken in every tree that splits on it (7.1% of bagging's splits lie in the block), and since the trees are deep, a different branch leads to a different identity, with different trees redirected to different identities. Boosting classifies it as identity 39 on both the clean and masked images, with a very low winning weight (0.091 on the masked face, with q for the true identity 0.057). Boosting's ensemble is fixed by sequential re-weighting, so its prediction does not change here. The training-weight evidence is weak: the average boosting weight of the true class is 0.025 (exactly uniform, $1/40$) and that of the predicted class is 0.030, so a heavier weighting of identity 39 contributes only a little. We therefore attribute the difference mainly to the diversity of independently grown trees in bagging.

4. Q3: Bias and variance as measured quantities

Protocol. We create 30 training sets by sampling, within each identity, six faces with replacement from that identity's six training faces (saved in `resamples.npy`). Each set contains on average $6[1-(5/6)^6] \approx 3.99$ distinct faces per identity. On each set we refit the selected single tree, the final bagged model ($B = 100$) and the selected boosting model without any tuning. For repetition r, $p_r(x_i)$ is the predicted class vector (single tree: `predict_proba`; bagging: vote fractions; boosting: normalised weighted vote q), $\bar{p}$ is its mean over the 30 repetitions and e_i the one-hot target. Then $\text{Bias}^2 = \frac{1}{N}\sum_i \|\bar{p}(x_i) - e_i\|_2^2$ and $\text{Var} = \frac{1}{NR}\sum_{i,r} \|p_r(x_i) - \bar{p}(x_i)\|_2^2$, with $N = 80$ clean test images.

4.1. (a) Selected models

The single tree has a large variance (0.630): the unlimited-depth tree changes a lot with the training set. Bagging reduces variance by a factor of about 26 (0.630 to 0.025) and raises accuracy from 0.488 to 0.775. Boosting has the smallest variance (0.017) but a large bias^2 (0.883) and the worst accuracy, with the largest spread between refits (± 0.082). **Caveat:** the tree's class vectors are almost one-hot, while bagging's and boosting's are soft (the mean winning weight is about 0.3 for bagging and 0.11 for boosting), so bias^2 and variance are not directly comparable between model types. In particular, a near-uniform vote vector has a small variance and a large bias^2 even when its arg-max is sometimes right, so boosting's variance is partly a consequence of its flat vote vectors, and its accuracy spread (0.082) is a better indicator of label stability.

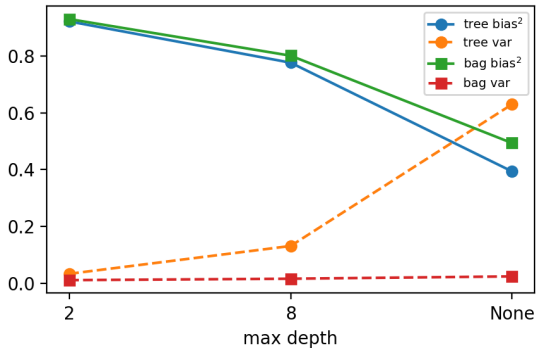


Figure 5. Bias^2 and variance of single trees and bagging versus maximum depth.

4.2. (b) Effect of depth

We repeat the calculation for single trees and 100-tree bagged ensembles at depths 2 (shallow), 8, and None (the selected depth, which is also the deepest possible; there is no deeper setting to test).

Depth. Going from depth 2 to unlimited depth, a single tree's bias^2 falls from 0.921 to 0.394 while its variance rises from 0.034 to 0.630 ($\approx 19\times$). Bagging removes a larger share of the variance at greater depth (66%, 88%, 96%) and in absolute terms (0.022, 0.115, 0.605). The bagged ensemble keeps almost the same bias^2 as the single tree at depths 2 and 8 and has higher bias^2 at depth None (0.493 vs. 0.394), because averaging vote vectors makes the output soft and spreads vote mass over several identities.

4.3. (c) Is the deepest tree where bagging helps most?

No. The largest variance reduction is at the deepest tree (0.605), but the largest accuracy gain is at depth 8 ($+0.308$ vs. $+0.287$ for None). The two gains are close, and with 80 test images and 30 refits we treat this as a modest difference, but it shows that variance removed is not the same as accuracy gained. Our measurements suggest why. (i) At depth 2 the trees are so weak that their bias^2 (0.921) dominates the error; bagging removes variance that was small to begin with (0.034) and accuracy stays at 0.152. Disagreement is the highest here (0.961), so the trees vote in nearly every direction. (ii) At depth 8 each tree still has a large variance (0.132) and leaf occupancy is 24.8 images per leaf, so bag-

	Single tree					Bagging (100)				Bagging effect	
Depth	Acc	Bias^2	Var	Samples/leaf	1-ex. leaves	Acc	Bias^2	Var	Disagr.	Var removed	Acc gain
2	0.066	0.921	0.034	78.67	0.000	0.152	0.929	0.011	0.961	0.022 (66%)	+0.086
8	0.172	0.776	0.132	24.82	0.000	0.480	0.801	0.016	0.926	0.115 (88%)	+0.308
None	0.488	0.394	0.630	6.00	0.000	0.775	0.493	0.025	0.807	0.605 (96%)	+0.287

Table 9. Single trees versus 100-tree bagged ensembles at three depths, averaged over 30 refits. "Disagr." is the average pairwise disagreement between bagged trees on the test set.

Model	Clean	Masked	Bias^2	Var
Single tree	0.488	0.343	0.394	0.630
Bagging	0.775	0.505	0.493	0.025
Boosting	0.350	0.293	0.883	0.017

Table 10. Mean test accuracy over the 30 refits on the clean and masked test sets, with clean-test bias^2 and variance.

ging has a lot of variance to remove while the bias is not yet worse than the single tree (0.776 vs. 0.801). (iii) At the unlimited depth, bagging removes the most variance (0.605), but the single tree has already reached a much better accuracy (0.488) and so has less room to improve; its leaf occupancy of 6.00 means about 40 leaves for 240 images, i.e. roughly one leaf per identity, and bagged-tree disagreement falls to 0.807, meaning the trees are more alike and the ensemble has less diversity to exploit. Also, bagging raises bias^2 at this depth. Fully grown trees contain almost no single-example leaves in this experiment (fraction 0.000 at every depth) because each resampled set contains duplicated images, so a leaf cannot isolate one example. The depth at which bagging helps most is therefore set by the balance between variance, bias and tree diversity, and not simply by the deepest tree.

4.4. (d) Final comparison

Most stable. Bagging. Its variance is 26 times smaller than the tree's, it has the smallest accuracy spread across refits (± 0.034) and the best accuracy on both test sets. Boosting has the smallest variance (0.017) but the largest accuracy spread (± 0.082), so its low variance reflects flat, low-confidence vote vectors more than stable decisions.

Most sensitive to the missing region. By absolute accuracy loss, bagging is most sensitive ($0.775 \rightarrow 0.505$, a drop of 0.270, or 35%), the single tree is next (0.145, 30%) and boosting is least (0.057, 16%). Bagging loses more because it has more to lose: when the central block is zeroed, trees with a split on those pixels take a different branch and end in different leaves (pairwise disagreement 0.92 on the face in Q2(d)), so the plurality weakens. For the single frozen tree of Q1, the loss was much larger (0.562 to 0.075), showing that the drop of one tree depends strongly on the training set, which the average over 30 refits smooths out. Boosting's small loss comes from a low starting point, not from robustness.

Does the lowest variance imply the best accuracy? No. Boosting has the lowest variance (0.017) yet the worst clean accuracy (0.350), because its bias^2 is the highest (0.883). Bagging has a slightly higher variance (0.025) but clearly the best accuracy (0.775). Error depends on both terms, and variance alone cannot rank models; moreover, our bias and variance values are measured on soft vectors whose scale differs between models.

5. Limitations and reproducibility

The validation and test sets contain only 80 images, so differences below about two images are not reliable; there is a single train/validation/test split; the boosting selection lies on the edge of the grid; and bias^2 and variance on soft vectors are not directly comparable across model families. All code, the saved split (`split.npz`), configurations (`q1_config.json`, `q2_config.json`), resamples, logs and the environment file are in the repository. To reproduce:

```
pip install -r requirements.txt
python q1.py > q1_log.txt 2>&1
python q2.py > q2_log.txt 2>&1
python q3.py > q3_log.txt 2>&1
```

References

[1] L. Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.

[2] J. Zhu, H. Zou, S. Rosset, T. Hastie. Multi-class AdaBoost. *Statistics and Its Interface*, 2(3):349–360, 2009.

[3] Y. Freund, R. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *JCSS*, 55(1):119–139, 1997.

[4] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR*, 12:2825–2830, 2011.